

Bitácora de Desarrollo – Franco

Proyecto: DonCE-Kong-Jr

Semana 1 – [10 de Noviembre – 16 de Noviembre]

Tareas Asignadas

- ☒ Configurar entorno de desarrollo con SDL3
- ☒ Crear ventana básica del juego
- ☒ Configurar CMake para MinGW-x64
- ☐ Implementar renderizado de sprites
- ☐ Diseñar arquitectura del cliente

Progreso Diario

Viernes 15 de Noviembre, 2025

Horas trabajadas: ~3 horas

Actividades realizadas:

- Creación de la ventana del juego usando SDL3
- Inicialización del renderer con presentación lógica de 512x448 píxeles
- Implementación del game loop principal con manejo de eventos
- Configuración de límite de ~60 FPS con `SDL_Delay(16)`
- Corrección de configuración de CMake para trabajar con MinGW-x64
- Limpieza de archivos de build antiguos (configuraciones de Visual Studio)
- Actualización de tareas en `CMakeLists.txt` para compilador GCC

Problemas encontrados:

- CMake estaba configurado para Visual Studio en lugar de MinGW
- Archivos de build `.vcxproj` y configuraciones de MSVC causaban conflictos
- Múltiples archivos binarios antiguos (`Debug/main.exe`, `.pdb`, `.dll`) generaban confusión

Soluciones implementadas:

- Reconfiguración completa de CMake para usar MinGW Makefiles
- Eliminación de 70+ archivos relacionados con configuración de Visual Studio
- Actualización de compiler flags en CMakeLists.txt
- Generación exitosa de main.exe con MinGW-x64

Aprendizajes:

- SDL3 utiliza `SDL_CreateWindowAndRenderer()` para inicializar ambos componentes simultáneamente
- `SDL_SetRenderLogicalPresentation()` permite mantener aspecto ratio correcto con letterboxing
- CMake puede generar diferentes tipos de build systems (Visual Studio, MinGW, etc.)
- Importancia de limpiar archivos de build al cambiar de toolchain

Pendientes para siguiente sesión:

- Cargar y renderizar sprites/imágenes
 - Implementar sistema de assets (imágenes, fuentes, sonidos)
 - Crear estructura básica del personaje jugador
 - Investigar manejo de input con SDL3
-

Semana 2 - [17 de Noviembre - 23 de Noviembre]

Tareas Asignadas

- ☒ Implementar renderizado de sprites
- ☒ Crear menú principal
- ☒ Agregar imagen de fondo
- ☒ Implementar movimiento del jugador
- ☒ Agregar animaciones del jugador
- ☐ Implementar lógica de enemigos

Progreso Diario

Martes 19 de Noviembre, 2025

Horas trabajadas: ~4 horas

Actividades realizadas:

- Implementación del menú principal con opciones interactivas
- Sistema de navegación por teclado (flechas arriba/abajo, Enter para seleccionar)

- Agregada imagen de fondo del nivel (bg.bmp - 58KB)
- Implementación de jugador movable con controles WASD
- Sistema básico de colisión y límites de pantalla
- Renderizado de sprites BMP con SDL_LoadBMP y SDL_CreateTextureFromSurface

Problemas encontrados:

- Gestión manual de memoria con SDL_FreeSurface después de crear texturas
- Coordenadas del jugador necesitaban limitarse a los bordes de la pantalla
- Integración del menú con el estado del juego

Soluciones implementadas:

- Implementación de enum para estados del juego (MENU, PLAYING, GAME_OVER)
- Sistema de máquina de estados para transiciones
- Liberación correcta de recursos SDL después de crear texturas
- Clamp de posición del jugador usando condicionales

Aprendizajes:

- SDL_LoadBMP es útil para cargar imágenes BMP sin dependencias adicionales
- Importancia de gestión de memoria en C (FreeSurface después de CreateTexture)
- Uso de enums para manejar estados del juego de forma limpia
- Renderizado de texto básico usando rectángulos coloreados como placeholders

Pendientes para siguiente sesión:

- Agregar más sprites de personajes
- Implementar sistema de animación
- Preparar arquitectura para comunicación con servidor

Sábado 23 de Noviembre, 2025

Horas trabajadas: ~2 horas

Actividades realizadas:

- Actualización del To Do List con tareas pendientes
- Revisión de arquitectura del proyecto
- Planificación de modularización del código

Pendientes para siguiente sesión:

- Refactorizar código en módulos separados
- Agregar todos los sprites del juego
- Implementar sistema de animación completo

Semana 3 - [24 de Noviembre - 30 de Noviembre]

Tareas Asignadas

- ☒ Refactorizar y modularizar el proyecto
- ☒ Agregar todos los sprites del juego
- ☒ Implementar sistema de animación del jugador
- ☒ Crear módulo de networking con sockets
- ☒ Inicializar comunicación básica con servidor
- ☐ Implementar protocolo de comunicación completo
- ☐ Sincronizar estado del juego entre cliente y servidor

Progreso Diario

Lunes 25 de Noviembre, 2025

Horas trabajadas: ~6 horas

Actividades realizadas:

Sesión Tarde (16:42 - 16:58):

- Agregados todos los sprites del juego (11 archivos BMP):
 - Personajes: dk-jr.bmp (28KB), dk.bmp (24KB), mario.bmp (5KB)
 - Enemigos: gator-blue.bmp, gator-red.bmp (2KB c/u)
 - Objetos: cage.bmp, fruit.bmp, life-icon.bmp, point-tally.bmp, points.bmp
- Agregada fuente kongtext.ttf (10KB) para UI
- Implementación de sistema de animación por frames para el jugador
- 4 estados de animación: idle, corriendo, saltando, escalando
- Sistema de spritesheet con SDL_Rect para selección de frames
- Timer de animación para controlar velocidad de frames

Sesión Noche (20:35 - 21:04):

- Refactorización completa del proyecto en arquitectura modular
- Creación de 10 módulos separados (.h/.c):
 - `assets.c/h` : Gestión de carga de recursos
 - `player.c/h` : Lógica y renderizado del jugador
 - `enemy.c/h` : Sistema de enemigos
 - `game_state.c/h` : Estado global del juego
 - `game_logic.c/h` : Lógica principal del juego

- `input.c/h` : Manejo de entrada del teclado
- `renderer.c/h` : Sistema de renderizado
- `network.c/h` : Comunicación por sockets
- Implementación del módulo de networking:
 - Inicialización de Winsock2 (WSAStartup)
 - Creación de socket TCP (AF_INET, SOCK_STREAM)
 - Función de conexión al servidor
 - Funciones send/receive básicas
 - Manejo de errores con WSAGetLastError()
- Actualización de CMakeLists.txt para compilar todos los módulos
- Enlace con biblioteca ws2_32 para sockets en Windows
- Corrección de inicialización de socket en main.c
- Integración del sistema de red con el game loop

Problemas encontrados:

- Código monolítico en main.c (~500 líneas) difícil de mantener
- Falta de separación de responsabilidades
- Inicialización incorrecta de sockets (faltaba WSAStartup antes de socket())
- Error de compilación por falta de enlace con ws2_32.lib
- Gestión de memoria compleja con múltiples recursos SDL

Soluciones implementadas:

- Arquitectura modular con separación clara de responsabilidades
- main.c reducido a ~77 líneas (solo inicialización y game loop)
- Cada módulo con responsabilidad única y bien definida
- CMakeLists.txt actualizado con todos los archivos fuente
- Agregado `-lws2_32` a las opciones de enlace
- Corrección del orden de inicialización: WSAStartup → socket() → connect()
- Sistema de asset manager para centralizar carga de recursos
- Structs bien definidas para Player, Enemy, GameState

Aprendizajes:

- Arquitectura modular en C requiere cuidadosa gestión de headers y dependencias
- Winsock2 requiere inicialización explícita antes de usar sockets
- CMake necesita listar explícitamente todos los archivos fuente
- Importancia de separar lógica de presentación desde el inicio
- Sockets en Windows requieren enlazar con ws2_32
- SDL_Rect permite implementar spritesheets de forma eficiente
- Forward declarations en headers previenen dependencias circulares

Pendientes para siguiente sesión:

- Implementar protocolo de comunicación cliente-servidor
 - Definir estructura de mensajes (handshake, input, estado)
 - Sincronizar posición del jugador con el servidor
 - Implementar lógica de enemigos
 - Sistema de colisiones completo
 - Manejo de desconexión y reconexión
-

Miércoles 26 de Noviembre, 2025

Horas trabajadas: ~5 horas

Actividades realizadas:

- Implementación casi completa del cliente con nuevas pantallas
- Mejora significativa en la conectividad con el servidor
- Sistema de estados del juego expandido:
 - `GAME_STATE_MENU` : Menú principal
 - `GAME_STATE_CONNECTING` : Pantalla de conexión como jugador
 - `GAME_STATE_CONNECTING_SPECTATE` : Conexión como espectador
 - `GAME_STATE_PLAYING` : Jugando activamente
 - `GAME_STATE_SPECTATING` : Modo espectador
 - `GAME_STATE_SPECTATE` : Selección de juego a observar
- Implementación del protocolo de comunicación:
 - Cliente envía "play" al conectar como jugador
 - Cliente envía "spectate X" para espiar juego X
 - Formato de estado: `STATE | gameNum | PLAYER | data | ENEMIES | data | FRUITS | data`
- Parsing de estado del juego desde el servidor:
 - Posición y estado del jugador (x, y, vidas, puntos)
 - Lista de enemigos con tipo y posición
 - Lista de frutas/coleccionables

Problemas encontrados:

- Sincronización de estados entre cliente y servidor
- Formato de mensajes inconsistente
- Manejo de conexiones no bloqueantes

Soluciones implementadas:

- Socket configurado como no bloqueante para evitar bloqueos
- Parseo robusto de mensajes con separadores definidos
- Prefijo de longitud de 2 bytes (formato Java writeUTF) para mensajes

Aprendizajes:

- Importancia de definir un protocolo de comunicación claro desde el inicio
 - Sockets no bloqueantes requieren manejo especial de errores `WSAEWOULDBLOCK`
 - Diseño de máquina de estados facilita gestión de múltiples modos de juego
-

Jueves 27 de Noviembre, 2025

Horas trabajadas: ~4 horas

Actividades realizadas:

Sesión Tarde (16:37):

- Correcciones varias en el sistema de comunicación
- Ajustes en el manejo de estados del juego
- Mejoras en la estabilidad de la conexión

Sesión Noche (18:42):

- Corrección mayor de errores en todo el sistema:
 - **Fix: Parsing de enemigos y frutas:** Cambio de `%d` a `%f` en `sscanf`
 - El servidor envía coordenadas como floats (ej: `100.0, 50.0`)
 - El cliente intentaba parsear como enteros, causando fallos
 - **Fix: Renderizado de enemigos:** Corrección de índices de frames en spritesheet
 - **Fix: Colección de frutas:** Corrección de `remove(i)` a `remove(i.intValue())`
 - En Java, `ArrayList.remove(Integer)` busca el objeto, no el índice
 - Esto causaba que las frutas dieran puntos infinitos sin desaparecer
- Implementación del patrón Observer para espectadores:
 - Interfaz `GameObserver` con método `onGameStateUpdate(String gameState)`
 - Interfaz `GameSubject` con `addObserver`, `removeObserver`, `notifyObservers`
 - Clase `SpectatorHandler` implementa `GameObserver`
 - Clase `Logic` implementa `GameSubject`
- Separación de lógica de espectadores de `GameClientHandler`
- Espectadores ahora reciben actualizaciones automáticamente vía Observer

Problemas encontrados:

- Enemigos y frutas no aparecían en el cliente
- Frutas daban puntos infinitos al recogerlas
- Espectadores no recibían actualizaciones del juego correcto
- `NullPointerException` en `clientsIDs` del servidor

Soluciones implementadas:

- Parsing con floats (`%f`) en lugar de integers (`%d`)
- Uso de `remove(i.intValue())` para eliminar por índice en ArrayList
- Patrón Observer para desacoplar espectadores del handler de jugadores
- Inicialización de `Integer clientsIDs = 0` en lugar de null

Aprendizajes:

- Java distingue entre `remove(int index)` y `remove(Object o)` en ArrayList
 - El patrón Observer es ideal para notificar múltiples clientes de cambios de estado
 - Importancia de consistencia entre formatos de datos cliente-servidor
 - Los wrappers de Java (Integer, Double) requieren atención especial con null
-

Viernes 28 de Noviembre, 2025

Horas trabajadas: ~3 horas

Actividades realizadas:

- Finalización del sistema de espectadores con patrón Observer
- Correcciones finales en el parsing de frutas:
 - Asegurar que todas las secciones usen `%f` para coordenadas
- Pruebas de integración cliente-servidor completas
- Verificación de funcionalidad:
 - ☒ Jugadores pueden conectarse y jugar
 - ☒ Espectadores pueden observar juegos en curso
 - ☒ Enemigos se renderizan y mueven correctamente
 - ☒ Frutas aparecen, se recolectan y desaparecen
 - ☒ Puntuación y vidas se sincronizan correctamente
- Documentación del protocolo de comunicación

Estado final del sistema:

- Cliente C/SDL3 completamente funcional
- Servidor Java con soporte para múltiples juegos
- Sistema de espectadores usando patrón Observer
- Comunicación bidireccional estable

Archivos clave modificados:

- `game_logic.c` : Parsing de estado, conexión, comandos
- `GameClientHandler.java` : Loop del juego, construcción de estado
- `Logic.java` : Implementación de GameSubject, colisiones
- `SpectatorHandler.java` : Implementación de GameObserver

- `Server.java` : Routing de jugadores vs espectadores
 - `App.java` : Registro de handlers y observers
-

Resumen General del Proyecto

Componentes Desarrollados

- **Cliente (C con SDL3):**
 - Sistema de renderizado con ventana 512x448
 - Arquitectura modular (10 módulos)
 - Sistema de animación por frames
 - Manejo de input con teclado
 - Módulo de networking con Winsock2
 - Menú principal funcional
 - Movimiento y animación del jugador
- **Servidor (Java):** En desarrollo
- **Manager (C):** Pendiente

Tecnologías y Librerías Utilizadas

- **SDL3:** Renderizado gráfico, manejo de ventanas, eventos, texturas
- **Winsock2:** Comunicación por sockets TCP en Windows
- **CMake:** Sistema de build con MinGW-x64
- **GCC:** Compilador C (MinGW)
- **Git:** Control de versiones

Arquitectura del Cliente

Módulos implementados:

1. `main.c` - Punto de entrada y game loop principal
2. `assets.c/h` - Gestión de recursos (imágenes, fuentes)
3. `player.c/h` - Lógica del jugador y animaciones
4. `enemy.c/h` - Sistema de enemigos
5. `game_state.c/h` - Estado global del juego
6. `game_logic.c/h` - Lógica principal del juego
7. `input.c/h` - Procesamiento de entrada

8. `renderer.c/h` - Sistema de renderizado
9. `network.c/h` - Comunicación cliente-servidor

Desafíos Principales

1. **Configuración del entorno de desarrollo:** Migración de Visual Studio a MinGW-x64, limpieza de archivos de configuración conflictivos
2. **Gestión de memoria en C:** Correcta liberación de superficies y texturas SDL
3. **Arquitectura modular:** Refactorización de código monolítico a sistema modular con dependencias claras
4. **Networking en Windows:** Inicialización correcta de Winsock2 y gestión de sockets
5. **Sistema de animación:** Implementación de spritesheets y control de frames

Logros Destacados

1.  Entorno de desarrollo completamente funcional con SDL3 y CMake
2.  Sistema de renderizado con presentación lógica y letterboxing
3.  Arquitectura modular bien estructurada (main.c reducido de 500 a 77 líneas)
4.  Sistema de animación por frames funcional
5.  Módulo de networking completo con protocolo definido
6.  Todos los sprites del juego integrados (11 archivos)
7.  Menú principal con navegación por teclado
8.  Comunicación cliente-servidor bidireccional funcional
9.  Sistema de espectadores con patrón Observer
10.  Sincronización de jugador, enemigos y frutas
11.  Parsing robusto de mensajes del servidor
12.  Sistema de estados del juego completo (menú, jugando, esperando)

Conclusiones y Reflexiones

- La modularización temprana es crucial para mantener el código manejable en proyectos C
- SDL3 ofrece una API limpia y potente para desarrollo de juegos 2D
- La gestión manual de memoria en C requiere disciplina y atención constante
- Windows requiere configuración específica para networking (Winsock2, ws2_32.lib)
- La separación de responsabilidades facilita enormemente el debugging y mantenimiento
- El uso de CMake con MinGW proporciona un flujo de trabajo consistente
- La implementación de sistemas de estado simplifica el manejo de la lógica del juego

Próximos Pasos

1. Pulir la experiencia de usuario (transiciones, feedback visual)

2. Agregar efectos de sonido y música
 3. Implementar sistema de niveles progresivos
 4. Mejorar IA de enemigos
 5. Agregar más tipos de coleccionables
 6. Implementar tabla de puntuaciones
 7. Optimizar rendimiento de red
-

Notas Adicionales

- **Total de commits hasta 28/11/2025:** 23+ commits en rama franco-develop-2
- **Líneas de código:** ~2,500+ líneas (cliente C + servidor Java)
- **Assets:** 11 sprites BMP + 1 fuente TTF
- **Tamaño del ejecutable:** ~224 KB (main.exe)
- **Patrones de diseño implementados:** Observer (espectadores), Factory (enemigos/frutas), State (estados del juego)
- **Protocolo de comunicación:** TCP con mensajes formato Java writeUTF (2-byte length prefix)

Bitácora de Desarrollo – Kevin

Proyecto: DonCE-Kong-Jr

Fase 1 – Implementación de Sockets y Comunicación

Tareas Asignadas

- ☒ Creación de Sockets en Java
- ☒ Implementación del sistema de comunicación cliente-servidor
- ☒ Desarrollo de clases de lógica del juego
- ☒ Creación de subclases para enemigos y coleccionables
- ☒ Mejoras en la comunicación con clientes

Progreso Detallado

Commit: c25dcaf - [ADD] Sockets java

Horas trabajadas: 8 horas

Actividades realizadas:

- Creación de la clase `Server.java` para gestionar conexiones TCP
- Implementación de la clase `Client.java` para conexiones de clientes
- Desarrollo de `ClientListener.java` para escuchar mensajes de clientes
- Creación de `Listener.java` interfaz para escuchadores
- Implementación de `Sender.java` para envío de mensajes

Problemas encontrados:

- Manejo de múltiples conexiones simultáneas
- Sincronización de threads en servidor

Soluciones implementadas:

- Uso de `ServerSocket` para aceptar conexiones en puerto 2121
- Implementación de threads separados para cada cliente

- Uso de `BufferedReader/PrintWriter` para comunicación

Aprendizajes:

- Arquitectura cliente-servidor en Java
- Manejo de excepciones de I/O
- Patrones de comunicación asincrónica

Pendientes para siguiente sesión:

- Integración con lógica del juego
 - Pruebas de estabilidad bajo carga
-

Commit: 6956555 - [ADD] Clases para lógica del juego

Horas trabajadas: 10 horas

Actividades realizadas:

- Creación de la clase `Entity.java` como entidad base
- Implementación de `Player.java` con mecánicas de movimiento y gravedad
- Desarrollo de `Platform.java` para plataformas del juego
- Creación de `Vine.java` para las lianas del juego
- Implementación de `Logic.java` como gestor principal del juego
- Desarrollo de colisiones y física del juego

Problemas encontrados:

- Complejidad en el manejo de colisiones
- Gravedad y movimiento de entidades
- Detección de intersecciones

Soluciones implementadas:

- Uso de rectángulos (`Rectangle`) para detección de colisiones
- Sistema de velocidad (`vx, vy`) para movimiento
- Método de colisiones comprobando todas las entidades

Aprendizajes:

- Estructuras de datos para entidades del juego
- Física básica en 2D
- Patrón de componentes para entidades

Pendientes para siguiente sesión:

- Refinamiento de detección de colisiones
 - Balanceo de mecánicas
-

Commit: ee27cdb - [ADD] Subclases para enemigos y coleccionables

Horas trabajadas: 8 horas

Actividades realizadas:

- Creación de `Enemy.java` como clase base para enemigos
- Implementación de `EnemyFactory.java` (patrón Factory)
- Desarrollo de `RedEnemy.java` y `RedFactory.java`
- Creación de `BlueEnemy.java` y `BlueFactory.java`
- Implementación de `Collectible.java` para frutas
- Desarrollo de `CollectibleFactory.java` y subclases:
 - `Banana.java` y `BananaFactory.java`
 - `Orange.java` y `OrangeFactory.java`
 - `Strawberry.java` y `StrawberryFactory.java`

Problemas encontrados:

- Diseño del patrón Factory
- Diferenciación de tipos de enemigos

Soluciones implementadas:

- Uso del patrón Factory para crear instancias
- Herencia de clases para enemigos y frutas
- Atributos específicos para cada tipo (valor de puntos, velocidad)

Aprendizajes:

- Patrón Factory en Java
- Herencia y polimorfismo
- Gestión de múltiples tipos de entidades

Pendientes para siguiente sesión:

- Comportamiento específico de enemigos
 - Animaciones y efectos
-

Commit: a6caa5e - [UPDATE] Cambios a clases del juego

Horas trabajadas: 6 horas

Actividades realizadas:

- Refinamiento de clases de entidades
- Mejora de métodos de colisión
- Actualización de atributos públicos
- Optimización de lógica de movimiento

Problemas encontrados:

- Inconsistencias en atributos
- Accesibilidad de variables

Soluciones implementadas:

- Estandarización de atributos públicos para acceso directo
- Mejora de métodos de actualización
- Refinamiento de lógica de física

Aprendizajes:

- Diseño de APIs públicas en Java
- Encapsulación vs acceso directo

Pendientes para siguiente sesión:

- Refactorización de acceso a variables
-

Commit: e26ab1a - [UPDATE] Cambios a lógica del juego

Horas trabajadas: 7 horas

Actividades realizadas:

- Implementación de gestión de múltiples instancias de juego
- Creación de `App.java` como clase principal
- Desarrollo de sistema de comandos CLI
- Mejora de manejo de colisiones
- Integración con sistema de sockets

Problemas encontrados:

- Gestión de dos juegos simultáneos
- Sincronización de estados

Soluciones implementadas:

- Clase App con dos instancias Logic independientes
- Sistema de comandos para debugging
- Métodos separados para cada juego

Aprendizajes:

- Patrón singleton para servidores
- Gestión de estado compartido
- Creación de interfaces de usuario CLI

Pendientes para siguiente sesión:

- Pruebas de sincronización
 - Handling de desconexiones
-

Commit: 9231e25 - [UPDATE] Envío de datos a clientes

Horas trabajadas: 8 horas

Actividades realizadas:

- Implementación de `GameClientHandler.java` para manejar clientes individuales
- Desarrollo de serialización de estado del juego
- Creación de protocolo de comunicación
- Envío de actualizaciones de juego a clientes

Problemas encontrados:

- Serialización de objetos complejos
- Latencia de comunicación
- Sincronización de estado

Soluciones implementadas:

- Conversión a strings para serialización simple
- Updates periódicos del estado
- Buffers para mensajes

Aprendizajes:

- Protocolos de comunicación en juegos
- Serialización de datos
- Manejo de buffers en red

Pendientes para siguiente sesión:

- Compresión de datos
 - Optimización de ancho de banda
-

Commit: 530f322 - [UPDATE] Cambios para comunicación con clientes

Horas trabajadas: 6 horas

Actividades realizadas:

- Refinamiento del sistema de comunicación
- Mejora de protocolos de mensajes
- Optimización de envío de datos
- Debugging de problemas de conexión

Problemas encontrados:

- Pérdida de conexión ocasional
- Desincronización entre cliente-servidor

Soluciones implementadas:

- Sistema de heartbeat para verificar conexión
- Resincronización de estado
- Mejor manejo de excepciones

Aprendizajes:

- Robustez en comunicación de red
- Detección de fallos
- Recuperación de errores

Pendientes para siguiente sesión:

- Testing exhaustivo
 - Optimización de performance
-

Mejoras Recientes (Noviembre 2025)

Actividad: Actualización de Java 21

Horas trabajadas: 1 hora

Actividades realizadas:

- Actualización de configuración de IntelliJ a Java 21 LTS

- Modificación de archivo `.idea/misc.xml`
- Verificación de compatibilidad de código

Problemas encontrados:

- Proyecto inicialmente en Java 18

Soluciones implementadas:

- Actualización de `languageLevel` a `JDK_21`
- Actualización de `project-jdk-name` a `21`

Aprendizajes:

- Compatibilidad hacia atrás de Java 21
 - Actualización de configuración del IDE
-

Actividad: Mejora en Recolección de Frutas

Horas trabajadas: 2 horas

Actividades realizadas:

- Implementación de eliminación de frutas recolectadas
- Refactorización del loop de colisiones de coleccionables
- Cambio de parámetros de `deletion`

Problemas encontrados:

- Frutas no se eliminaban después de recolección
- Parámetros inconsistentes entre creación y eliminación

Soluciones implementadas:

- Cambio a loop con índice inverso para eliminación segura
- Método `deleteCollectible(int vineIdx, double y)` en `Logic`
- Método `handleDeleteFruit()` en `App` para comandos CLI
- Uso de mismos parámetros (`vinIdx`, `y`) para creación y eliminación

Aprendizajes:

- Iteración segura al eliminar de `ArrayLists`
- Consistencia de APIs
- Facilidad de uso en interfaces CLI

Pendientes para siguiente sesión:

- Testing de eliminación de frutas
 - Validación de parámetros
-

Resumen General del Proyecto

Tecnologías y Librerías Utilizadas

- **Sockets de Java** - Comunicación cliente-servidor
- **BufferedReader/PrintWriter** - Manejo de I/O
- **ArrayList** - Gestión de colecciones
- **Rectangle (AWT)** - Detección de colisiones

Desafíos Principales

1. **Comunicación en red confiable** - Sincronización entre cliente-servidor
2. **Gestión de múltiples instancias** - Dos juegos simultáneos
3. **Física y colisiones** - Detección precisa de intersecciones
4. **Patrón Factory** - Crear diferentes tipos de entidades dinámicamente

Logros Destacados

1. **Sistema de sockets funcional** - Comunicación TCP establecida
2. **Lógica de juego completa** - Movimiento, colisiones, puntuación
3. **Patrón Factory implementado** - 6 subclases de enemigos y frutas
4. **Interfaz CLI para debugging** - Comandos para crear y eliminar entidades

Conclusiones y Reflexiones

- La arquitectura cliente-servidor proporciona una base sólida para el juego multijugador
 - El patrón Factory se ha demostrado muy útil para extensibilidad
 - La física del juego es relativamente simple pero efectiva para Donkey Kong Jr
 - La comunicación de red requiere manejo robusto de excepciones y desconexiones
 - Java 21 mantiene excelente compatibilidad con el código existente
-

Notas Adicionales

- Se han mantenido commits pequeños y descriptivos con prefijos [ADD], [UPDATE], [FIX]
- La rama kevin-develop se ha mantenido sincronizada con cambios progresivos

- El proyecto está bien estructurado en paquetes: `game` , `sockets`
- Se recomienda implementar logging más robusto para debugging futuro

Bitácora de Desarrollo – Pamela Chacón

Proyecto: DonCE-Kong-Jr

Nota: Comencé a trabajar en el proyecto a partir del 15 de noviembre de 2025.

Semana 1 – [15 de Noviembre – 23 de Noviembre]

Tareas Asignadas

- ☒ Configurar entorno de desarrollo
- ☒ Familiarizarse con SDL3 y arquitectura del proyecto
- ☒ Búsqueda y preparación de assets gráficos
- ☒ Implementación de sprites en formato BMP
- ☒ Documentación del proyecto

Progreso Diario

Viernes 15 de Noviembre, 2025

Horas trabajadas: ~3 horas

Actividades realizadas:

- Configuración del entorno de desarrollo con SDL3
- Estudio de la arquitectura cliente-servidor del proyecto
- Revisión de la documentación de SDL3 para renderizado de sprites
- Análisis del código base existente (main.c, renderer.c)
- Configuración de MinGW-x64 para compilar el cliente

Problemas encontrados:

- Curva de aprendizaje con SDL3 (cambios desde SDL2)
- Entendimiento de la estructura modular del cliente
- Configuración inicial de CMake y dependencias

Soluciones implementadas:

- Estudio de la documentación oficial de SDL3
- Revisión de ejemplos de código del equipo
- Colaboración con Franco para entender la arquitectura

Aprendizajes:

- SDL3 utiliza nuevos métodos como `SDL_CreateWindowAndRenderer()`
- Importancia de la presentación lógica para mantener aspecto ratio
- Arquitectura cliente-servidor con sockets TCP/IP
- Diferencias entre SDL2 y SDL3

Pendientes para siguiente sesión:

- Comenzar con la búsqueda de assets gráficos
 - Preparar sprites en formato BMP
 - Familiarizarse más con el sistema de renderizado
-

Sábado 16 de Noviembre, 2025

Horas trabajadas: ~4 horas

Actividades realizadas:

- Búsqueda y selección de sprites para el juego
- Investigación sobre sprites de Donkey Kong Jr original
- Estudio de paletas de colores retro para el juego
- Análisis de resoluciones y tamaños de sprites apropiados

Problemas encontrados:

- Encontrar sprites que coincidan con la estética del juego original
- Determinar resoluciones apropiadas para cada elemento
- Compatibilidad de formatos de imagen con SDL3

Soluciones implementadas:

- Decisión de usar formato BMP por compatibilidad con SDL3
- Establecer estándar de 16x16 píxeles para enemigos y frutas
- Definir 32x16 píxeles para el jugador principal

Aprendizajes:

- SDL3 soporta nativamente archivos BMP con `SDL_LoadBMP()`
- Importancia de mantener consistencia en resoluciones
- Consideraciones de paleta de colores para estética retro

Pendientes para siguiente sesión:

- Convertir y preparar sprites en formato BMP
 - Crear spritesheets para animaciones
 - Organizar assets en la estructura de carpetas
-

Domingo 17 de Noviembre, 2025

Horas trabajadas: ~5 horas

Actividades realizadas:

- Creación de sprites en formato BMP
- Preparación del spritesheet del jugador (dk-jr.bmp - 448x16, 14 frames)
- Creación de sprites de enemigos (gator-red.bmp, gator-blue.bmp)
- Diseño del spritesheet de frutas (fruit.bmp - 48x16, 3 frames)
- Creación de elementos UI (life-icon.bmp, points.bmp, point-tally.bmp)
- Organización de assets en carpeta assets/img/

Problemas encontrados:

- Mantener consistencia en el estilo pixel art
- Alineación correcta de frames en spritesheets
- Tamaño de archivos BMP sin comprimir

Soluciones implementadas:

- Uso de grilla para mantener alineación perfecta de píxeles
- Verificación de dimensiones con herramientas de edición
- Optimización de paletas de colores para reducir tamaño

Aprendizajes:

- Técnicas de pixel art para sprites de 16x16
- Organización de spritesheets para animaciones fluidas
- Formato BMP y sus características (sin comprimir)

Pendientes para siguiente sesión:

- Integrar sprites adicionales (fondo, decoraciones)
 - Probar carga de sprites en el juego
 - Ajustar detalles visuales según necesidad
-

Semana 2 - [18 de Noviembre - 24 de Noviembre]

Tareas Asignadas

- ☒ Completar todos los assets gráficos
- ☒ Integrar fuente personalizada
- ☒ Preparar documentación de assets
- ☒ Soporte en testing de renderizado
- ☒ Crear sprites adicionales (logo, decoraciones)

Progreso Diario

Lunes 18 de Noviembre, 2025

Horas trabajadas: ~3 horas

Actividades realizadas:

- Creación del fondo del nivel (bg.bmp - 224x256 píxeles)
- Diseño de elementos decorativos (Donkey Kong, jaula)
- Creación de sprites dk.bmp y cage.bmp
- Revisión de todos los assets para consistencia visual

Problemas encontrados:

- Mantener coherencia artística entre todos los elementos
- Optimización de tamaño del archivo de fondo
- Detalles visuales para elementos más grandes

Soluciones implementadas:

- Uso de paleta de colores consistente en todos los sprites
- Compresión visual manteniendo calidad pixel art
- Revisión iterativa con el equipo

Aprendizajes:

- Diseño de fondos para juegos retro
- Optimización de assets sin perder calidad visual
- Importancia de la coherencia visual en el proyecto

Pendientes para siguiente sesión:

- Crear logo del juego
 - Buscar e integrar fuente personalizada
 - Documentar todos los assets creados
-

Martes 19 de Noviembre, 2025

Horas trabajadas: ~4 horas

Actividades realizadas:

- Diseño del logo del juego (logo.bmp - 183x56 píxeles)
- Búsqueda e integración de la fuente Kong Text (kongtext.ttf)
- Creación del sprite de Mario (mario.bmp - 32x16, 2 frames)
- Organización final de la carpeta assets/
- Pruebas de carga de assets con Franco

Problemas encontrados:

- Encontrar una fuente que combine con la estética del juego
- Integración de SDL_ttf para renderizado de texto
- Tamaño apropiado de fuente (8px) para resolución del juego

Soluciones implementadas:

- Selección de Kong Text como fuente oficial del proyecto
- Configuración de SDL_ttf en el sistema de assets
- Establecer tamaño de fuente a 8 píxeles para mejor legibilidad

Aprendizajes:

- Uso de SDL3_ttf para renderizado de fuentes TrueType
- Importancia de tipografía en la experiencia de usuario
- Carga y gestión de recursos con SDL3

Pendientes para siguiente sesión:

- Documentar especificaciones de cada asset
- Crear tabla de referencia de sprites
- Preparar documentación para README

Miércoles 20 de Noviembre, 2025

Horas trabajadas: ~3 horas

Actividades realizadas:

- Creación de documentación detallada de assets
- Elaboración de tabla con especificaciones de cada sprite
- Contribución a la sección de Assets del README
- Verificación de integridad de todos los archivos BMP

Problemas encontrados:

- Organizar información de forma clara y accesible
- Documentar convenciones de naming de archivos
- Especificar formato de spritesheets

Soluciones implementadas:

- Creación de tabla con nombre, dimensiones y descripción
- Documentación de estructura de carpetas assets/
- Especificación de frames por spritesheet

Aprendizajes:

- Importancia de documentación clara para trabajo en equipo
- Convenciones de naming para assets
- Organización estructurada de recursos del proyecto

Pendientes para siguiente sesión:

- Soporte en testing de sistema de renderizado
 - Ajustes visuales según feedback del equipo
 - Revisión de integración de assets en el código
-

Jueves 21 de Noviembre, 2025

Horas trabajadas: ~2 horas

Actividades realizadas:

- Testing de carga de assets en el cliente
- Verificación de renderizado correcto de sprites
- Ajustes menores en assets según feedback
- Coordinación con Franco sobre sistema de animación

Problemas encontrados:

- Algunos sprites no se visualizaban correctamente
- Ajustes en alineación de frames de animación
- Verificación de transparent color key

Soluciones implementadas:

- Revisión y corrección de dimensiones de sprites
- Ajuste de color key para transparencia
- Verificación de carga con SDL_LoadBMP

Aprendizajes:

- Debugging de carga de assets en SDL3
- Manejo de transparencia en sprites BMP
- Coordinación efectiva con el equipo de desarrollo

Pendientes para siguiente sesión:

- Continuar con soporte en testing
 - Preparar assets adicionales si son necesarios
 - Documentar proceso de creación de assets
-

Semana 3 - [25 de Noviembre - 28 de Noviembre]

Tareas Asignadas

- ☒ Testing exhaustivo del cliente
- ☒ Soporte en debugging visual
- ☒ Documentación final del proyecto
- ☒ Revisión de integración de assets
- ☒ Preparación de presentación

Progreso Diario

Lunes 25 de Noviembre, 2025

Horas trabajadas: ~4 horas

Actividades realizadas:

- Testing exhaustivo del sistema de renderizado
- Verificación de animaciones de jugador (idle, walk, jump, climb)
- Testing de sprites de enemigos y frutas
- Pruebas de menú principal y UI
- Revisión de fuente Kong Text en diferentes contextos

Problemas encontrados:

- Algunos frames de animación no fluían correctamente
- Timing de animaciones necesitaba ajustes
- Renderizado de texto en diferentes resoluciones

Soluciones implementadas:

- Coordinación con Franco para ajustar timing de animación
- Verificación de índices de frames en spritesheets
- Testing de escalado de fuente

Aprendizajes:

- Importancia de testing exhaustivo en sistemas gráficos
- Debugging visual requiere atención al detalle
- Trabajo colaborativo en resolución de problemas

Pendientes para siguiente sesión:

- Continuar con testing de features adicionales
 - Documentar resultados de testing
 - Preparar assets para presentación
-

Martes 26 de Noviembre, 2025

Horas trabajadas: ~3 horas

Actividades realizadas:

- Testing de networking y sincronización visual
- Verificación de renderizado en modo espectador
- Pruebas de múltiples clientes simultáneos
- Revisión de sistema de vidas y puntuación visual
- Testing de pantallas de game over y credits

Problemas encontrados:

- Sincronización visual entre clientes
- Renderizado correcto de estado de juego
- Visualización de información de múltiples jugadores

Soluciones implementadas:

- Verificación de protocolo de comunicación
- Testing de actualización de sprites según estado del servidor
- Coordinación con equipo para ajustes

Aprendizajes:

- Complejidad de sincronización en juegos multijugador
- Importancia de feedback visual inmediato
- Testing de casos edge en networking

Pendientes para siguiente sesión:

- Finalizar testing de todas las features
 - Preparar documentación final
 - Revisar README con información completa
-

Miércoles 27 de Noviembre, 2025

Horas trabajadas: ~4 horas

Actividades realizadas:

- Revisión completa del README del proyecto
- Actualización de sección de Assets con información detallada
- Documentación de especificaciones técnicas de sprites
- Creación de tabla de assets con dimensiones y formatos
- Verificación de links y referencias en documentación

Problemas encontrados:

- Mantener documentación consistente y actualizada
- Formateo correcto de tablas en Markdown
- Organización lógica de la información

Soluciones implementadas:

- Revisión estructurada sección por sección
- Uso de formato Markdown apropiado
- Verificación de exactitud técnica de información

Aprendizajes:

- Importancia de documentación clara para proyectos
- Buenas prácticas en documentación técnica
- Uso avanzado de Markdown para documentación

Pendientes para siguiente sesión:

- Testing final de la build completa
 - Ajustes finales según feedback
 - Preparación para entrega
-

Jueves 28 de Noviembre, 2025

Horas trabajadas: ~5 horas

Actividades realizadas:

- Testing final del proyecto completo
- Verificación de build en Windows con MinGW
- Pruebas de todas las funcionalidades (jugar, espectral, game over)
- Revisión final de todos los assets
- Implementación de icono de ventana usando life-icon.bmp
- Testing de sprite flipping (jugador y enemigos)
- Verificación de animación de salto corregida
- Documentación final de cambios realizados

Problemas encontrados:

- Animación de salto tenía jitter en el pico del salto
- Faltaba icono de aplicación en la ventana
- Algunos detalles visuales finales

Soluciones implementadas:

- Corrección de lógica de animación de salto en servidor
- Implementación de `SDL_SetWindowIcon()` con life-icon.bmp
- Revisión y ajustes finales de renderizado

Aprendizajes:

- Debugging de animaciones frame-perfect
- Configuración de iconos de aplicación en SDL3
- Testing exhaustivo previo a entrega

Logros del día:

-  Proyecto completamente funcional
 -  Todos los assets integrados correctamente
 -  Animaciones fluidas sin jitter
 -  Icono de aplicación implementado
 -  Documentación completa y actualizada
-

Resumen del Proyecto

Contribuciones Principales

Assets Gráficos (11 sprites BMP + 1 fuente):

- `bg.bmp` (224x256) - Fondo del nivel principal
- `dk-jr.bmp` (448x16) - Spritesheet del jugador (14 frames de 32x16)
- `dk.bmp` (24 KB) - Sprite de Donkey Kong
- `mario.bmp` (32x16) - Spritesheet de Mario (2 frames)
- `gator-blue.bmp` (32x16) - Enemigo cocodrilo azul (2 frames)
- `gator-red.bmp` (32x16) - Enemigo cocodrilo rojo (2 frames)
- `cage.bmp` (6 KB) - Jaula decorativa
- `fruit.bmp` (48x16) - Spritesheet de frutas (3 frames: naranja, banana, fresa)
- `life-icon.bmp` (8x8) - Icono de vida / icono de aplicación
- `point-tally.bmp` (7 KB) - Contador de puntos
- `points.bmp` (2 KB) - Indicador de puntos
- `logo.bmp` (183x56) - Logo del juego
- `kongtext.ttf` (10 KB) - Fuente Kong Text

Documentación:

- Documentación detallada de assets en README
- Especificaciones técnicas de sprites
- Esta bitácora de desarrollo
- Tablas de referencia de recursos

Testing y QA:

- Testing exhaustivo del sistema de renderizado
- Verificación de animaciones y sprites
- Testing de networking visual
- QA de experiencia de usuario

Estadísticas del Proyecto

- **Total de horas trabajadas:** ~40 horas
- **Assets creados:** 12 archivos gráficos + 1 fuente
- **Días de trabajo activo:** 14 días (15 nov - 28 nov)
- **Commits contribuidos:** Integrados en branch develop
- **Bugs encontrados y reportados:** ~8
- **Features testeadas:** Todas las funcionalidades del cliente

Tecnologías Utilizadas

- **Herramientas de diseño:** Editores de pixel art
- **Formato de assets:** BMP (sin comprimir)

- **Fuentes:** TrueType Font (TTF)
- **Testing:** SDL3 en Windows con MinGW-x64
- **Documentación:** Markdown
- **Control de versiones:** Git/GitHub

Aprendizajes Clave

1. Diseño de Assets para Juegos:

- Técnicas de pixel art y sprite design
- Creación de spritesheets para animaciones
- Optimización de assets para rendimiento

2. SDL3 y Renderizado:

- Carga de recursos con `SDL_LoadBMP()`
- Integración de SDL3_ttf para fuentes
- Sistema de renderizado y presentación lógica

3. Trabajo en Equipo:

- Coordinación efectiva en proyecto multi-lenguaje
- Comunicación clara de especificaciones técnicas
- Iteración basada en feedback del equipo

4. Testing y QA:

- Metodologías de testing para aplicaciones gráficas
- Debugging visual y resolución de problemas
- Verificación de integración de componentes

5. Documentación Técnica:

- Creación de documentación clara y estructurada
- Uso de Markdown para documentación técnica
- Importancia de documentar especificaciones

Reflexión Final

Este proyecto me permitió trabajar en la creación completa de assets para un juego con arquitectura cliente-servidor, aprendiendo sobre diseño gráfico para videojuegos, integración de recursos en SDL3, y la importancia del trabajo colaborativo en proyectos complejos. La experiencia de crear todos los sprites desde cero y ver cómo cobran vida en el juego fue muy gratificante.

El trabajo en conjunto con Franco en el cliente y Kevin en el servidor demostró la importancia de una comunicación clara y documentación detallada. Cada asset creado tuvo que ser especificado con precisión en dimensiones, formato y propósito para integrarse correctamente en el sistema de renderizado.

Los desafíos encontrados, especialmente en la creación de spritesheets con alineación perfecta y el testing de animaciones, me enseñaron la importancia de la atención al detalle en el desarrollo de videojuegos. Ver el proyecto final funcional, con todos los elementos visuales trabajando en armonía, hizo que el esfuerzo valiera la pena.